

CS3210 Revision Notes (L02-L06)

Oct 1, 2026 · @zi shen

Spend most of your time on synchronization patterns, Amdahl vs Gustafson calculations, Foster's methodology with task graphs, and the CUDA execution and memory model; the rest is mostly definitions. Ranking is by slide weight and the L06 learning outcomes, since I don't know your exam format.

Study priorities

Tier 1 is where you can be asked to compute or derive something; Tier 2 needs precise definitions and comparisons; Tier 3 is recall.

Tier	Topic	Lecture	What "in depth" means here
1	Semaphore solutions (producer-consumer, readers-writers, lightswitch, turnstile)	L02	Trace interleavings; explain why a variant deadlocks or starves; write them from memory
1	Race, data race, critical section, deadlock (4 conditions), livelock, starvation	L02	Give a minimal example of each and distinguish them precisely
1	Lock implementation: why naive spinlock fails, test-and-set	L02	Explain the atomicity argument; spinlock vs mutex trade-off
1	Amdahl, Gustafson, speedup, efficiency, cost-optimality	L05	Derive both, compute numbers, state assumptions and when each applies
1	Foster's methodology, task dependence graph, degree of concurrency	L04	Compute critical path and concurrency; count communications for a grid decomposition
1	CUDA execution model: grid/block/warp, divergence, occupancy	L06	Pick and justify a launch configuration; compute occupancy from resource limits
1	CUDA memory: coalescing, shared memory banks, host-device transfer	L06	Count transactions for an access pattern; spot bank conflicts

Tier	Topic	Lecture	What "in depth" means here
2	CPU time equation, AMAT (multi-level), MIPS/MFLOPS flaws	L05	Plug-in calculations; explain why MIPS is misleading
2	Flynn's taxonomy; UMA/NUMA/ccNUMA/COMA; distributed vs shared memory	L03	Compare with trade-offs; map programming model to hardware
2	Cache coherence problem; spatial/temporal locality; false sharing and padding	L03, L05	Walk the stale-read scenario; fix a layout with padding
2	Parallel patterns (fork-join, parbegin-parend, SPMD, master-worker, task pool, pipeline)	L04	Pick the pattern for a scenario and justify it
2	Data vs task parallelism; shared address space vs message passing	L04	Decompose the same loop both ways
3	Bit-level, ILP (pipeline, superscalar, SIMD), SMT, multicore designs	L03	Definitions and examples
3	Process vs thread, user vs kernel threads, thread mappings, fork/exec	L02	Definitions and trade-offs
3	GPU history, compute capability, compilation (PTX, SASS)	L06	Recall only

L02 Processes, threads, synchronization

Parallelization pipeline

Sequential algorithm → **decompose** into tasks (programmer) → **schedule** tasks onto processes/threads (OS and libraries) → **map** processes/threads onto physical cores (OS). The same three steps reappear in L04 as Foster's methodology.

Process vs thread

	Process	Thread
Address space	Own (exclusive)	Shared with its process
Private state	PC, registers, stack, heap, globals, OS resources (files, sockets)	PC, SP, registers, own runtime stack only

	Process	Thread
Communication	Explicit (IPC through the OS: shared memory, message passing, pipes, signals)	Implicit through shared memory
Creation cost	High: syscall, allocate and copy data structures	Lower: no address-space copy
Context switch	Costly	Cheaper
Failure isolation	Strong	None: one bad thread can corrupt all

Memory layout of a process: text, data (globals), heap, stack. With threads, text/data/heap are shared and each thread gets its own stack. A thread's stack exists only while the thread is active.

fork, exec, wait

- `fork()` creates a child that is a copy of the parent's address space and resumes at the instruction after `fork`. It returns **0 in the child** and the **child's PID in the parent**.
- `exec` replaces the process image with a new program; `fork` + `exec` is how a shell launches programs.
- `exit(status)` ends a process; the parent uses `wait` / `waitpid` to collect it.
- Web-server idiom: parent `accept()`s, forks a child to serve the client, and the parent closes its copy of the socket.
- Multiprogramming: context switch saves/restores state (overhead). Time slicing gives pseudo-parallelism; true parallel execution needs multiple cores.
- Exceptions are **synchronous** (caused by the instruction: divide by zero, bad address); interrupts are **asynchronous** (timer, keyboard).

Threads

Type	Who manages	Pros	Cons
User-level	Thread library; OS unaware	Fast switching	No parallelism across cores; one blocking I/O call blocks the whole process
Kernel	OS	True multicore use; one thread blocking doesn't block others	Heavier operations

Mappings: **many-to-one** (all user threads on one kernel entity, library schedules), **one-to-one** (each user thread has a kernel thread, OS schedules), **many-to-many** (library assigns user threads to a pool of kernel threads; the mapping can change over time).

POSIX pattern: `pthread_create` to start, `pthread_join` to wait. Without the join, `main` may `exit` and kill all threads. Choose the thread count to match application parallelism and available cores, but not so many that creation/management overhead dominates.

Why synchronization is needed

Threads interleave arbitrarily and at different speeds; scheduling is outside program control. Synchronization restricts the allowed interleavings.

Bank example: `withdraw` does read balance, subtract, write balance. Two threads each withdrawing \$100 from \$1000 can both read 1000 and both write 900: one withdrawal is lost.

Definitions to keep distinct:

- **Race condition:** outcome depends on timing/interleaving of concurrent execution.
- **Data race** (a type of race condition): two concurrent accesses to the same location, no protection, at least one is a write.
- **Critical section:** code that touches shared state and needs mutual exclusion.
- **Mutual exclusion:** at most one thread inside the critical section; others wait on entry; when one leaves, another may enter.

Mechanisms

- **Locks:** `acquire()` / `release()`, must be paired. A lock either **spins** (spinlock, busy-waits) or **blocks** (mutex, thread sleeps).

- **Semaphores** (Dijkstra, THE system, 1968): non-negative integer with atomic `wait()` /P (decrement, block if it would go below 0) and `signal()` /V (increment, wake a waiter). Binary semaphore (count 1) acts as a mutex; counting semaphore (count N) lets N threads through. Which waiting thread wakes after a `signal` is undefined.
 - Drawbacks: global-variable-like, no link between semaphore and the data it guards, same primitive used for mutual exclusion and for scheduling/coordination, easy to misuse.
- **Monitors:** high-level, mutex plus condition variables, needs language support (Java `synchronized`, `wait`, `notify`).
- **Barrier:** no thread proceeds past the barrier until all have arrived.
- **Messages:** synchronization via data transfer over a channel; maps naturally to distributed systems.

Deadlock, starvation, livelock

Deadlock: every process in a set waits for an event only another process in the set can cause. It occurs only if all four hold:

1. **Mutual exclusion:** a resource is held non-sharably.
2. **Hold and wait:** a process holds one resource while waiting for another.
3. **No pre-emption:** resources can't be forcibly taken.
4. **Circular wait:** P1 waits for P2, ..., Pn waits for P1.

Four responses: ignore it, **prevent** (break one of the four conditions), **avoid** (control allocation, e.g. refuse unsafe grants), **detect and recover** (find a cycle in the wait-for graph).

Starvation: a process never makes progress because others keep getting the resource; a side effect of scheduling or lock fairness (high-priority always wins; one thread always wins the lock).

Livelock: states keep changing but nobody progresses (two people stepping aside in a corridor, the slide's picture). Unlike deadlock, nobody is blocked; they are busy but useless. Special case of starvation.

Producer-consumer (know all four versions)

Variables: `mutex = Semaphore(1)`, `items = Semaphore(0)`.

Producer <code>event = waitForEvent()</code> <code>mutex.wait()</code> <code> buffer.add(event)</code> <code>mutex.signal()</code> <code>items.signal()</code>	Consumer <code>items.wait()</code> <code>mutex.wait()</code> <code> event = buffer.get()</code> <code>mutex.signal()</code> <code>event.process()</code>
---	---

- The **first version** signals `items` inside the mutex. Correct but slightly wasteful: a woken consumer immediately blocks on the mutex the producer still holds. Moving `items.signal()` outside the mutex (shown above) is the improvement.
- **Broken version:** consumer does `mutex.wait()` **before** `items.wait()`. If the buffer is empty the consumer holds the mutex while sleeping on `items`; the producer can never get the mutex to add an item. **Deadlock** (hold and wait + circular dependency). Lesson: block on the counting semaphore first, take the mutex last and briefly.
- **Finite buffer:** add `spaces = Semaphore(buffer_size)`. Producer: `spaces.wait()` then mutex section then `items.signal()`. Consumer: `items.wait()`, mutex section, then `spaces.signal()`. Two counting semaphores track full and empty slots; the mutex protects the buffer structure.

Readers-writers

Any number of readers may be inside together; a writer needs exclusive access.

```

readers = 0; mutex = Semaphore(1); roomEmpty = Semaphore(1)

Writer:
roomEmpty.wait()
    # write
roomEmpty.signal()
locks

Reader:
mutex.wait()
    readers += 1
    if readers == 1: roomEmpty.wait()    # first in
mutex.signal()
    # read
mutex.wait()
    readers -= 1
    if readers == 0: roomEmpty.signal() # last out
unlocks
mutex.signal()

```

Lightswitch abstracts the reader pattern: first thread in turns the "light" on (waits on the semaphore), last out turns it off (signals it). Then readers call

`readSwitch.lock(roomEmpty) / unlock(roomEmpty)`.

Problem: writers starve while readers keep arriving. **Fix: turnstile** (`Semaphore(1)`). A writer takes the turnstile and holds it while waiting for the room; readers pass through it briefly (`wait` then `signal`) before entering, so once a writer is waiting, new readers queue behind it. No-starve version:

```

Writer:
turnstile.wait()
roomEmpty.wait()
# write
turnstile.signal()
roomEmpty.signal()

Reader:
turnstile.wait()
turnstile.signal()
readSwitch.lock(roomEmpty)
# read
readSwitch.unlock(roomEmpty)

```

Writer-priority version uses two lightswitches (`writeSwitch` on `noReaders` , `readSwitch` on `noWriters`) plus a `noReaders` gate readers must pass first, so waiting writers block new readers entirely. Trade-off: readers can now starve.

Implementing locks

1. Naive spinlock: `while (lock->held); lock->held = 1;` . **Broken:** two threads can both see `held == 0` before either sets it (a context switch between the test and the set). The lock implementation itself has a critical section: the recursion problem.
2. Fix: make acquire **atomic** with hardware help: an atomic instruction such as **test-and-set** (record old value, set flag, return old value, all indivisible), or disable/enable interrupts (prevents context switches; single-core only).
3. `acquire: while (test_and_set(&lock->held));` and `release: lock->held = 0;` .
4. **Spinlock cost:** waiters burn CPU. On a uniprocessor the holder cannot run while another thread spins, so spinning is pure waste; the holder lost the CPU through yield/sleep or an involuntary context switch.
5. All higher-level synchronization (semaphores, monitors) is built on atomicity.

L03 Architecture and memory

Two layers of parallelism: **application parallelism** (decompose the problem into tasks) and **hardware parallelism** (cores and processors that execute them). Architecture matters because it determines how much of the application parallelism you can actually use and how expensive memory access is.

Concurrency vs parallelism. Concurrency: tasks start, run and finish in overlapping time periods, possibly interleaved on one core. Parallelism: tasks execute at the exact same instant. Parallel implies concurrent; the reverse is false.

Levels of hardware parallelism

Level	Mechanism	Key facts	Limit or catch
Bit	Wider word (16 → 32 → 64 bit: 8086 1978, 80386 1985, Pentium 4/Opteron 2003)	A 64-bit add on 64-bit values needs 1 operation; on a 32-bit machine it needs 2	Gains saturate once words are wide enough
Instruction: pipelining	Split execution into stages (IF, ID, EX, WB); different instructions in different stages each cycle	Parallelism across time ; max speedup = number of stages	Data/control hazards, bubbles, dependencies; fixed by speculation and out-of-order execution (watch read-after-write)
Instruction: superscalar	Duplicate pipelines; several instructions through the same stage per cycle	Parallelism across space ; IPC up, CPI down; scheduling is dynamic (hardware) or static (compiler); e.g. an i7 core: 14 stages, 6 micro-ops per cycle	Structural hazards; instructions come from one thread; only about 2-3 independent instructions in typical code
Instruction: SIMD	More ALUs, one instruction broadcast to all	SSE: 128-bit xmm registers (4 floats); AVX: 256-bit (8 floats / 4 doubles)	Poor for divergent control flow
Thread (SMT)	Hardware thread contexts (own PC, registers) sharing the core's units; e.g. Intel Hyper-Threading, 2 threads per core	Two logical cores per physical core; issues one scalar instruction per clock from one of the threads	Threads share execution resources, so not 2x
Processor	More cores / more processors	Needs multiple independent execution flows in the software	Memory system and synchronization become the bottleneck

End of ILP scaling: clock rate stopped increasing and ILP offers no further benefit, so extra transistors went into more cores. This is the motivation for the whole course.

Flynn's taxonomy (1972)

Classified by number of instruction streams (one PC = one stream) and data streams.

Class	Meaning	Examples / notes
SISD	One instruction stream, each instruction on one datum	Classic uniprocessor
SIMD	One instruction stream, many data	Vector processors (1980s supercomputers), SSE/AVX, GPUs; exploits data parallelism; bad at divergence
MISD	Many instruction streams on the same data	Essentially none; systolic arrays at best
MIMD	Each unit fetches its own instructions and has its own data	Most multiprocessors today

A GPU's streaming multiprocessors are a **variant**: within a group of threads it is effectively SIMD (same code), and across groups it is effectively MIMD.

Multicore designs

- **Hierarchical**: cores share cache levels, size grows toward the root (private L1, shared L2/L3, shared external memory). Desktops, servers, GPUs.
- **Pipelined**: data flows through a chain of cores, each doing one step; suits router/network processors and graphics.
- **Network-based**: cores with local caches/memory linked by an interconnect (Sun Niagara 2). Trend: **network on chip** (bandwidth, scalability, fault tolerance, energy efficiency, lower memory access time).

Memory: latency, bandwidth, and why it dominates

- **Latency**: time to service one request (e.g. 100 cycles/100 ns). **Bandwidth**: data rate (e.g. 20 GB/s). A processor **stalls** when the next instruction depends on data not yet available.
- Caches reduce effective latency and supply high bandwidth.
- Slide example: a core doing one add per clock but needing a 64-byte load for every few adds keeps the memory system 100% busy; the core idles waiting. **Bandwidth is the critical resource.**

- Consequences for program design: reuse data already loaded (temporal locality), share data between threads, and prefer recomputing over storing and reloading ("math is free"). Programs must access memory infrequently.

Memory organization of parallel computers

Distributed memory (multicomputers): each node has processor, cache, and **private** memory; nodes joined by an interconnection network; communication is explicit messages. Scales well; harder to program.

Shared memory (multiprocessors): all processors access one address space through a memory provider; the program is unaware of the physical layout. Advantages: no need to partition code/data, communication is efficient (no data movement). Disadvantages: needs explicit synchronization, and contention limits scalability. Classified on two axes:

Variant	Memory access time	Notes
UMA	Same for every processor	Single shared memory; contention limits it to small processor counts
NUMA	Local faster than remote	Physically distributed memories form one global address space ("distributed shared memory"); examples: AMD Ryzen, multi-socket servers
ccNUMA	NUMA plus caches kept coherent in hardware	Each node has cache to cut contention
COMA	Each memory block acts as a cache; data migrates dynamically under the coherence scheme	No fixed home for data
NCC (no cache coherence)	Software must manage	Mentioned as the alternative to CC

Hybrid (distributed-shared): clusters of shared-memory nodes, the common supercomputer layout; the model L08 builds toward.

Cache coherence

Problem: the same variable can sit in several caches. Slide scenario: memory holds `u = 5`; PU1 and PU3 both read it (cached `u = 5`); PU3 writes `u = 7` in its cache; PU1 (and PU2) then read `u` and see the stale 5. Requirement: after a local update, other processors must not see the old value. Hardware enforces it with a **cache coherence protocol** (cache coherence is about a single location; **memory consistency** is about the ordering of accesses to different locations, and the slides only name it).

Takeaways to be able to state

- Why ILP stopped scaling and why we moved to thread/processor-level parallelism.
- SIMD vs MIMD vs the GPU hybrid.
- UMA vs NUMA vs ccNUMA vs COMA vs distributed memory, with the scalability/programmability trade-off.
- Why bandwidth (not FLOPs) is the bottleneck and what code changes address it.

L04 Parallel programming models I

Parallelism and its limits

Parallelism = average number of units of work executable in parallel per unit time. It is limited by **program dependencies** (data, control) and **runtime overheads** (memory contention, communication, thread/process management, synchronization). Work = useful work + overhead from dependencies. Overheads can cost milliseconds (millions of flops), so given enough parallel work, **overhead is the biggest barrier to speedup**.

Data vs task parallelism

	Data parallelism	Task (functional) parallelism
Split	The data; each PU does similar work on its part	The work; each PU does a different task
Example: <code>a[i]=b[i]*c[i];</code> <code>d[i]=b[i]/c[i]</code>	PU1 takes the first half of <code>i</code> , PU2 the second half (both do mul and div)	PU1 computes all of <code>a</code> (mul), PU2 computes all of <code>d</code> (div)
Hardware fit	SIMD, vector, GPU	MIMD
Key question	Can the data be split (mostly) independently?	Can dependencies between tasks be minimized?

	Data parallelism	Task (functional) parallelism
Typical code	Independent-iteration loops (loop parallelism), OpenMP <code>parallel for</code>	Fork-join of different functions, database query tree

A loop is data-parallel only if iterations are independent. `a[i] = b[i-1] + c[i]` is fine (reads `b`, writes `a`); `a[i] = a[i-1] + c[i]` has a loop-carried dependency and is not.

SPMD (single program, multiple data) is the standard way to do data parallelism on MIMD: every core runs the same program and uses its index `me` ($0..p-1$) to pick its slice. Scalar product: each core sums its chunk, then partial sums are combined.

Task dependence graph (TDG)

A **directed acyclic graph**: node = task (value = expected execution time), edge = dependency. Used to evaluate a decomposition.

- **Critical path length**: longest weighted path = minimum possible completion time with unlimited processors.
- **Degree of concurrency** = total work / critical path length: average parallelism available.

Slide example (database query `Model=civic AND Year=2001 AND (Color=green OR white)`): four leaf lookups of 10 each, then combining tasks.

Decomposition	Total work	Critical path	Length	Degree of concurrency
A	63	Task 4 $\rightarrow$ 6 $\rightarrow$ 7 (10 + 9 + 8)	27	$63 / 27 = 2.33$
B	64	Task 1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7 (10 + 6 + 11 + 7)	34	$64 / 34 = 1.88$

Decomposition A is better: shorter critical path, higher concurrency, even though total work is similar. Degree of concurrency is an upper bound on useful processors for that decomposition; extra processors beyond it sit idle.

Models of coordination

Model	Communication	Structure	Matches hardware	Notes
Shared address space	Read/write shared variables; locks for mutual exclusion	Little structure; logical extension of uniprocessor programming	Shared memory (UMA, NUMA)	Not all accesses cost the same and that cost is invisible in the code; needs hardware support; contention and NUMA hurt scaling
Data parallel	Map a side-effect-free function over a collection; no communication between invocations	Very rigid	SIMD, GPU	Stream programming; CUDA, OpenCL, ISPC relax the strict structure
Message passing	Private address spaces; explicit send/receive (MPI)	Highly structured communication	Distributed memory (clusters, supercomputers)	No system-wide load/store needed, so commodity machines can be combined

Any model can be implemented on any hardware: message passing on shared memory (send = copy into library buffers, receive = copy out); shared address space on distributed memory in software (page-fault handler issues network requests, writes send invalidations), but less efficiently.

Representation of parallelism (how much the programmer must spell out): implicit (automatic parallelizing compilers, functional languages like Haskell) vs explicit (OpenMP: implicit scheduling; BSPLib: implicit communication but explicit mapping; MPI and Pthreads: explicit scheduling, mapping, communication and synchronization). Automatic parallelization struggles with pointers/indirect addressing, loops of unknown bounds, and opaque memory hierarchies, so mostly you do it yourself.

Foster's design methodology (PCAM)

Sequential algorithm → **Partitioning** → **Communication** → **Agglomeration** → **Mapping**. The first two aim to expose parallelism and the last two to fit the machine.

1. Partitioning: split computation and data into many small tasks.

- *Domain (data-centric) decomposition* = data parallelism: divide data evenly, then attach computations to data. 3-D grid options: 24 tasks of 3 points, 6 tasks of 12, or 1 task of 72.
- *Functional (computation-centric) decomposition* = task parallelism: divide computation, then attach data (climate model: atmosphere, ocean, hydrology, land surface exchanging wind velocity and sea-surface temperature).
- Rules of thumb: at least **10x more primitive tasks than cores**; avoid redundant computation and storage; tasks of roughly equal size; number of tasks grows with problem size.

2. Communication: determine data flow between tasks (this is the cost of parallelism).

- *Local*: a task needs data from a few neighbors; create channels. Example: 2-D five-point stencil, $X[i][j]^{(t+1)} = (4X[i][j] + X[i-1][j] + X[i+1][j] + X[i][j-1] + X[i][j+1]) / 8$ at time t .
- *Global*: many tasks contribute to one result (e.g. sum). Don't create channels early. The naive centralized sum over N tasks takes $O(N)$ because it is centralized and sequential; a tree-style reduction distributes the work and overlaps communication with computation.
- Rules: balanced communication, each task talks to few neighbors, communication proceeds in parallel, overlap computation with communication.

3. Agglomeration: merge tasks into larger ones; keep number of tasks $\geq$ number of cores. Goals: cut communication and task-creation cost, keep scalability, simplify programming. Rules: higher locality, task count still grows with problem size, suitable for target systems, code-change cost reasonable.

Granularity effect on an 8x8 grid (slide numbers; each task counted with 4 neighbors, send and receive):

Partition	Tasks	Data transfers
Fine: 1 point/task	$8 \times 8 = 64$	$64 \times 4 \times 2 = 512$
Coarse: 4x4 points/task	$2 \times 2 = 4$	$4 \times 4 \times 2 = 32$

The underlying principle is **surface-to-volume ratio**: computation scales with the area of a block, communication with its perimeter, so bigger blocks have less communication per unit of work. Counter-pressure: too few tasks hurts load balance and hides parallelism.

4. Mapping: assign tasks to cores. Conflicting goals: maximize utilization (spread tasks) vs minimize inter-processor communication (co-locate tasks that talk). Done by the OS on centralized multiprocessors, by the user on distributed-memory systems. Optimal mapping is **NP-hard**, so use heuristics; map neighboring tasks to cores that are directly connected in the topology. Rules: consider one-task-per-core and multiple-tasks-per-core designs; with dynamic allocation the allocator must not become a bottleneck; with static allocation use a task-to-core ratio of at least 10:1.

Parallel programming patterns

A pattern gives a coordination structure for tasks; they are not mutually exclusive.

Pattern	Idea	Implementation / example	Watch out for
Fork-join	Task creates independent children that may join at different times; children can run the same or different code	Processes, threads; database query as nested forks (P1 = civic AND 2001, P2 = green OR white, then join)	Join waits for the slowest child
Parbegin-parend	A construct creates a set of threads for a block of statements and waits for all at the end; all forks together, all joins together	OpenMP <code>#pragma omp parallel for (matrix multiply with shared(a,b,result) private(i,j,k))</code>	Implicit barrier at the end; shared vs private variable declarations
SIMD	Same instruction on all threads, in lockstep	SSE/AVX	Divergence

Pattern	Idea	Implementation / example	Watch out for
SPMD	Same program on different data; threads may diverge by <code>if</code> or core speed	GPU programs, MPI programs	No implicit synchronization; add it explicitly
Master-worker	Master initializes, assigns work, collects results, does I/O/timing; workers wait for instructions	MPI matrix multiply: rank 0 distributes <code>a</code> rows and <code>b</code> , workers compute <code>rows_per_worker = size / workers</code> rows	Master becomes a bottleneck
Task pool	Fixed set of threads pull tasks from a shared pool; tasks can add tasks; done when pool empty and all threads idle	Java <code>Executors.newFixedThreadPool(5)</code> with 10 tasks	Pool access must be synchronized; overhead matters for fine-grained tasks. Good for irregular/adaptive work; thread-creation cost independent of task count
Producer-consumer	Producers fill a shared buffer, consumers drain it	Java <code>synchronized</code> + <code>wait / notify</code> with <code>while</code> loops	Buffer full/empty handling; same semaphore logic as L02
Pipeline	Stream of data elements passes through stages <code>T1..Tp</code> ; each stage receives, processes, sends	Stream/functional parallelism	Throughput limited by the slowest stage; needs many elements to fill the pipe

Subtle points: in the producer-consumer code, `wait` sits in a `while`, not an `if`, because a woken thread must recheck the condition. Fork-join differs from `parbegin-parend` in that forks and joins may be staggered rather than all at once.

L05 Performance of parallel systems

Goals differ by audience: users want low **response time** (wall-clock time from start to end of a program); operators want high **throughput** (jobs or transactions per second). This lecture targets response time: first understand a sequential program, then how fast a parallel one can be.

Sequential response time

Response time = user CPU time + system CPU time (OS routines) + waiting time (I/O, and other programs under time sharing). Waiting depends on system load, system time on the OS.

$$T_{\text{user}}(A) = N_{\text{cycle}}(A) \times T_{\text{cycle}}, \quad T_{\text{cycle}} = \frac{1}{\text{clock rate}}$$

$$N_{\text{cycle}}(A) = \sum_{i=1}^n n_i(A) \times \text{CPI}_i \quad \rightarrow \quad T_{\text{user}}(A) = N_{\text{instr}}(A) \times \text{CPI}(A) \times T_{\text{cycle}}$$

- `N_instr` depends on the instruction set architecture and the compiler; `CPI` depends on CPU internals, memory system and compiler; `T_cycle` on the hardware.
- Poll question on the slides (why is the i7 faster than the Xeon on sequential code?): the answer is **clock frequency** (3+ GHz vs 2.2 GHz, so shorter cycle time), not core count or logical cores. Higher frequency means *shorter* time per instruction.

Adding memory access time (one-level cache):

$$T_{\text{user}}(A) = \big(N_{\text{instr}}(A) \times \text{CPI}(A) + N_{\text{rw_op}}(A) \times R_{\text{miss}}(A) \times N_{\text{miss_cycles}}\big) \times T_{\text{cycle}}$$

`N_rw_op` = number of reads and writes, `R_miss` = miss rate, `N_miss_cycles` = extra cycles to load a cache line. Cache terms: LLC = last-level cache, cache line/block = unit of memory held in cache, mapping = how a block is stored and located. Load: check cache; hit = deliver using the offset; miss = fetch block, allocate a line, load. Store: hit/miss behavior depends on write policy (write-back vs write-through) and miss policy (write-allocate vs write-around).

Average memory access time, applied recursively for multi-level caches and virtual memory:

$$T_{\text{read_access}} = T_{\text{read_hit}} + R_{\text{read_miss}} \times T_{\text{read_miss}}$$

$$T_{\text{L1_read_miss}} = T_{\text{L2_read_hit}} + R_{\text{L2_read_miss}} \times T_{\text{L2_read_miss}}, \quad R_{\text{global}} = R_{\text{L1_read_miss}} \times R_{\text{L2_read_miss}}$$

$T_{\text{read_hit}}$ is the time to access the cache regardless of hit or miss (extra time is charged to the miss term). The L2 miss rate here is the **local** rate (misses per L2 access); the global rate multiplies the two.

Throughput measures and their flaws

$$\text{MIPS}(A) = \frac{N_{\text{instr}}(A)}{T_{\text{user}}(A) \times 10^6} = \frac{\text{clock rate}}{\text{CPI}(A) \times 10^6}, \quad \text{MFLOPS}(A) = \frac{N_{\text{fl_ops}}(A)}{T_{\text{user}}(A) \times 10^6}$$

- **MIPS** counts only instruction count, ignores what each instruction does, and is easy to inflate (a compiler or ISA that emits more, simpler instructions raises MIPS while the program gets slower). Hence "meaningless indicator of performance".
- **MFLOPS** doesn't distinguish cheap from expensive floating-point ops; still used to rank the Top500; only meaningful when the goal is maximizing FLOP throughput.

Parallel metrics

$T_p(n)$ = time from start to end of the parallel program on p PUs, problem size n . It includes local computation, data exchange, synchronization, and waiting (unequal load, waiting for shared structures).

$$S_p(n) = \frac{T_{\text{best_seq}}(n)}{T_p(n)}, \quad C_p(n) = p \times T_p(n), \quad E_p(n) = \frac{T_{\text{best_seq}}(n)}{C_p(n)} = \frac{S_p(n)}{p}$$

- Theoretically $S_p \leq p$. **Superlinear speedup** ($S_p > p$) happens in practice, e.g. each core's working set now fits in cache, or one thread's loads warm a shared L3 for another.
- **Cost** = processor-time product = total work including idle time. A parallel program is **cost-optimal** if its cost equals the sequential time asymptotically (efficiency constant, ideally 1).
- **Efficiency 1** means ideal speedup $S_p = p$.
- Use the **best** sequential algorithm in the numerator, not the parallel code run on one core. Difficulties: the best algorithm may be unknown, the asymptotically optimal one may be slower in practice, or too complex to implement.

Amdahl's law (1967): fixed problem size

Let f be the sequential (non-parallelizable) fraction. Time on one core = T ; on p cores = $f T + (1 - f) T / p$.

$$S_p(n) = \frac{1}{f + \frac{1-f}{p}} \leq \frac{1}{f}$$

- Speedup is capped by $1/f$ however many processors you add: $f = 0.25$ gives at most 4; $f = 0.05$ gives at most 20.
- Relay-race intuition: total time is dominated by the slow runner who can't be split.
- Historical effect: discouraged building huge parallel machines and pushed research toward parallelizing compilers that shrink f .
- Also called fixed-workload (problem-constrained) scaling.

Gustafson's law (1988): scaled problem

Observation: f is **not constant**; as n grows, the serial part (initialization etc.) grows slowly, so $f(n)$ shrinks. Gustafson measured about 1000x speedup on about 1000 processors on real problems. Many problems are bound by runtime, not size (weather forecast: be as accurate as possible within a day), so we grow the problem.

Slide derivation: sequential part takes constant time τ_f ; parallelizable part takes $\tau_v(n, p) = (T^*(n) - \tau_f)/p$ assuming perfect parallelization with no overhead.

$$S_p(n) = \frac{\tau_f + \tau_v(n, 1)}{\tau_f + \tau_v(n, p)} = \frac{\frac{\tau_f}{T^*(n) - \tau_f} + 1}{\frac{\tau_f}{T^*(n) - \tau_f} + \frac{1}{p}} \quad ; \quad \lim_{n \rightarrow \infty} S_p \quad \text{if } T^*(n) \text{ grows strictly monotonically}$$

So $\lim_{n \rightarrow \infty} f(n) = 0$ gives $S_p \rightarrow p$: Amdahl's cap is circumvented for large problems. The slide's closed form is $S_p = p / (1 + (p-1) f(n))$.

Takeaways. Amdahl is pessimistic (same problem, f limits speedup); Gustafson is optimistic (bigger problem, near-perfect parallelism). They answer different questions and are not in contradiction. **Both assume:** f is known (fixed or zero), parallel parts are perfectly parallel with no overhead, identical processors, no communication cost growth when scaling, memory not a bottleneck (coherence, consistency, bandwidth). Know when to apply each and these limitations.

Scalability

- Fixed problem, growing machine: small problems are dominated by parallel overhead; problems sized for a big machine may not fit a small one (working set exceeds cache or memory, thrashing).

- Scaling constraints: **problem-constrained (PC)**: same problem faster (Amdahl); **time-constrained (TC)**: more work in fixed time (Gustafson); **memory-constrained (MC)**: largest problem that fits in memory. Application-oriented scaling parameters (particles per processor, transactions per processor) usually combine several numbers.

Arithmetic intensity, contention, locality

$$\text{Arithmetic intensity} = \frac{\text{amount of computation (e.g. instructions)}}{\text{amount of communication (e.g. bytes)}}$$

The reciprocal is the communication-to-computation ratio (the average bandwidth demand). Modern processors have far more compute than bandwidth, so you need **high** arithmetic intensity to keep cores busy. The memory-throughput slide shows loads stalling the pipeline and the memory bus saturating: at steady state core utilization is set by instruction throughput and memory throughput.

- **Contention**: many requests to one resource (memory, link, lock, server) in a short window; the resource is a hot spot.
- **Temporal locality / sharing**: schedule threads that use the same data on the same processor at the same time; reduces communication.
- **Spatial locality and cache lines**: when tasks on different cores share a cache line (e.g. a 6x6 or 4x4 tile in a row-major array with 4-element lines), cores contend for and invalidate each other's lines. Fix: choose tile sizes aligned to cache lines and **pad** the array so each task's data occupies whole lines; also lay out data to benefit from prefetching. (This is **false sharing** when the shared line holds *different* variables.)

Performance analysis procedure

Set the goal first (response time, throughput, speedup) and check your experiment matches it, since results are easily misleading or unrepresentative. Try the simplest parallel version first, measure, then find the bottleneck by perturbation:

Suspected bottleneck	Experiment	Interpretation
Instruction rate	Add more non-memory math	Time grows linearly with op count → compute-bound
Memory bandwidth/latency	Remove almost all math, keep the same loads	Little time drop → memory-bound
Locality	Change all array accesses to <code>A[0]</code>	Big speedup → poor locality / cache misses are the problem

Suspected bottleneck	Experiment	Interpretation
Synchronization	Remove atomics/locks (keeping similar work)	Big speedup → synchronization-bound

Worked numbers

Question	Working	Answer
CPU time: 2×10^9 instructions, CPI 1.5, 3 GHz	$2e9 \times 1.5 / 3e9$	1.0 s
AMAT: L1 hit 1 cycle, L1 miss rate 5%, L2 hit 10, L2 local miss rate 20%, memory 100	$T_{L1miss} = 10 + 0.2 \times 100 = 30$; $AMAT = 1 + 0.05 \times 30$	2.5 cycles; global miss rate 1%
Amdahl: $f = 0.1$, $p = 8$	$1 / (0.1 + 0.9/8) = 1 / 0.2125$	$S = 4.71$, $E = 0.59$; limit 10
Amdahl: $f = 0.05$, speedup of 10 needs	$0.05 + 0.95/p = 0.1$	$p = 19$
Amdahl: $f = 0.05$, $p = 1024$	$1 / (0.05 + 0.95/1024)$	$S = 19.6$ (the "20x" limit is approached, never reached)
Classic Gustafson: serial share of the <i>parallel</i> run $\alpha = 0.1$, $p = 8$	$S = p - \alpha (p - 1) = 8 - 0.7$	7.3

L06 GPGPU and CUDA

L06's four learning outcomes are the study checklist: (1) judge whether a problem suits GPU acceleration, (2) choose and justify a CUDA execution configuration, (3) design an implementation using the right memory and execution resources, (4) evaluate and optimize from architecture and measurements.

Why GPUs

Amdahl says low- f problems need many processing units ($f = 0.05$ needs about 1000 PUs to approach the 20x limit). CPUs have too few cores; adding nodes brings communication overhead and a different programming model (L08). A GPU on the same board, linked by PCIe/NVLink, is far faster to reach than another node.

Origin: graphics. A 1080p frame is $1920 \times 1080 \times 3$ = about 6.2 million values, 60+ times a second: a massive data-parallel problem. The same shader program runs over streams of vertices, fragments and pixels. GPGPU (CUDA, 2007; alternatives ROCm, OpenCL, OpenACC) generalizes this to AI, simulation, finance, etc.; most top Top500 systems use GPUs.

CPU vs GPU: CPU = few complex cores optimized for single-thread latency; GPU = many simpler cores optimized for **throughput**, hiding latency by switching to other ready threads when one stalls. A single GPU thread is weaker than a CPU thread (the slides compare a one-thread CPU matmul with a one-thread GPU matmul). The GPU is not standalone: the **host** (CPU, host memory) sends data and kernels to the **device** (GPU, device memory).

Suitability checklist (LO1). Good: large amounts of data-parallel work, independent elements, high arithmetic intensity, regular control flow, enough compute to amortize host-device transfer. Poor: mostly serial, heavy branching/divergence, irregular memory access, small data, low data reuse. Note the slide's point: it can be worth running a kernel on the GPU even with no kernel-level speedup if that avoids host-device copies.

CUDA programming model

- Function qualifiers: `__global__` = **kernel**, launched from host, runs on device, must return `void`, can't access host memory, no varargs. `__device__` = runs on device, called from device. `__host__` = CPU function (default).
- Launch: `kernel<<<blocks_per_grid, threads_per_block>>>(args)`. This launches a grid of thread blocks, and all threads run the kernel (SPMD). The launch is asynchronous; `cudaDeviceSynchronize()` waits for completion.
- Thread identity: `threadIdx` (within block), `blockIdx` (within grid), `blockDim`, `gridDim`; each can be 1D, 2D or 3D (programmer convenience only, **no hardware impact**).
- Compilation: `nvcc` sends host code to g++ and compiles device code to **PTX** (intermediate assembly), then to **SASS** (GPU machine code); linked with CUDA libraries.

Software	runs on	Hardware
thread	->	CUDA core (one lane)
thread block	->	streaming multiprocessor (SM), one SM for the whole kernel duration
grid (kernel)	->	whole GPU

Block rules: all threads of a block share shared memory and can synchronize (`__syncthreads()`); a block never migrates between SMs; blocks of a grid are **independent** (no synchronization between blocks except via cooperative groups) and can run in any order. This gives **transparent scalability**: the same kernel runs on a GPU with 2 SMs or 100 SMs, blocks are just scheduled over time.

Hardware (Hopper H100 as the example)

- GPU = many **SMs** sharing an L2 cache and GPU DRAM (slide: 144 SMs, 96 GB on the full die; connected by PCIe/NVLink/HBM).
- Each SM: 4 sub-units the slides call streaming processors (SPs); 128 FP32 cores, 64 FP64, 64 INT32, 4 tensor cores (matrix multiply-accumulate); a **huge register file** (up to 255 registers per thread vs about 16-32 on a CPU) so that warp context switches cost nothing: all resident warps' registers are on chip. L1/shared memory per SM.
- "CUDA core" in marketing is one lane/ALU (e.g. one INT32 unit), not a CPU-style core.
- Compute capability (CC) numbers the architecture generation (Ampere 8.x, Ada/Lovelace 8.9, Hopper 9.0, Blackwell 10.0); the SoC cluster has A100 (8.6 as listed), H100 and H200 (9.0).

Warps, SIMT, divergence

- A block is split into **warps** of 32 consecutive threads (first warp holds thread 0). 50 threads per block = 2 warps; the second has 18 active and 14 inactive lanes (wasted). Use multiples of 32.
- Warps execute **SIMT** (single instruction, multiple threads): all 32 threads start at the same PC and run in lockstep; one instruction per cycle for the warp. Each thread has its own registers and logical PC.
- **Divergence**: if threads of a warp take different branches, the hardware runs each path in turn with non-participating threads **masked off**, so throughput drops. Divergence only matters **within a warp**: a branch decided per whole warp (e.g. on `threadIdx.x / 32`) costs nothing; `threadIdx.x % 32` splits every warp in half. (Since Volta, independent thread scheduling relaxes strict lockstep but divergent paths still serialize in practice.)
- **Warp scheduling**: warps on an SM take turns; because registers stay resident, switching to another ready warp is immediate. This is how GPUs hide memory latency, so you want many resident warps.
- **Residency**: several blocks can be resident per SM (about 32 max). Register file is partitioned among resident threads and shared memory among resident blocks, so resource use per thread/block limits residency.

Memory model

Memory	Scope / lifetime	Speed	Notes
Registers	Per thread	Fastest	Up to 255/thread (Hopper)
Local	Per thread	Slow (it is GPU DRAM)	Spill when data doesn't fit in registers; large per-thread arrays
Shared (<code>__shared__</code>)	Per block	Fast (on-chip, part of L1)	Programmer-managed; needs <code>__syncthreads()</code>
Global	All threads and host	Slow DRAM, cached	Read/write; persists across kernels
Constant	All threads, read-only	Cached	Best for uniform/linear access; <code>cudaMemcpyToSymbol</code>
Texture	All threads, read-only	Cached	Best for 2-D spatial locality

API: `cudaMalloc(&p, nbytes)`, `cudaMemset`, `cudaFree`, `cudaMemcpy(dst, src, nbytes, cudaMemcpyHostToDevice | DeviceToHost | DeviceToDevice)`. **Unified memory**: declare `__managed__` and the runtime migrates data automatically (convenient, but you lose control over when copies happen). Synchronization: `__syncthreads()` is a **block-wide barrier** (all threads must reach it, so never put it inside a divergent branch); also `__syncwarp`, `cudaDeviceSynchronize`.

Optimization strategy (LO4)

Three levers: (1) memory bandwidth, (2) parallel execution/occupancy, (3) instruction throughput.

Host-device transfer. Device memory bandwidth far exceeds host-device bandwidth, so minimize transfers, **batch** many small copies into one large one, use **pinned (page-locked)** memory, zero-copy where useful, and overlap with `cudaMemcpyAsync` plus **streams** (copy for chunk $i+1$ while computing chunk i).

Coalescing. A warp's global accesses are combined into 32-byte transactions (sectors). Ideal: thread k reads word k of an aligned array, so $32 \text{ threads} \times 4 \text{ B} = 128 \text{ B} = 4 \text{ transactions}$, 100% utilization (any permutation inside the 128 B is fine). Misaligned start: 5 transactions. Stride of 8 floats (32 B) between threads: each thread touches its own sector, so $32 \text{ sectors} \times 32 \text{ B} = 1024 \text{ B}$ fetched for 128 B used = **12.5% utilization** (worst case). Rule: make consecutive `threadIdx.x` access consecutive addresses (arrange data layout and index mapping accordingly).

Shared memory banks. 32 banks, successive 32-bit words go to successive banks (word w is in bank $w \bmod 32$), each bank serves 32 bits per cycle. If two threads in a warp hit **different words in the same bank**, the accesses serialize (bank conflict); an n -way conflict takes n times as long. Same-word reads by several threads are broadcast (no conflict). Examples: thread i reads `s[i]` (no conflict), `s[2*i]` (2-way), `s[32*i]` (32-way, worst), all read `s[0]` (broadcast). Fix: pad (e.g. `float tile[32][33]`) so column accesses fall in different banks.

Execution configuration (LO2). Occupancy = active warps on an SM / maximum warps per SM. Need many more warps than SPs so a stalled warp can be replaced; too-low occupancy can't hide latency. But maximum occupancy isn't the goal: too many threads per block means fewer registers per thread, so spills to slow local memory. Prefer more blocks over huge blocks (more SMs working). Avoid several CUDA contexts per GPU (multiple processes sharing a GPU).

Resource limits shown on the slide (CC 10 table): 32 blocks/SM, 2048 threads/SM (= 64 warps), 65536 registers/SM and per block, 233472 B shared/SM, 49152 B shared/block (default), 1024 threads/block.

Config	Limiting factor	Resident threads/SM	Occupancy
256 thr/block, 32 regs/thread	threads: $2048/256 = 8$ blocks; regs: $65536/32 = 2048$	2048	100%
256 thr/block, 64 regs/thread	regs: $65536/64 = 1024$ threads = 4 blocks	1024	50%
32 thr/block (tiny blocks)	block cap: 32 blocks x 32	1024	50%
256 thr/block, 48 KB shared/block	smem: $233472/49152 = 4$ blocks	1024	50%

Instruction throughput. Prefer single precision; avoid integer division and modulo (use shifts/masks for powers of two); minimize intra-warp divergence.

Worked example: matrix multiply

One thread per output element; each block computes a tile of C . For size 400 and 16×16 blocks: `gridDim = ((400+15)/16, (400+15)/16) = (25, 25)`, 625 blocks of 256 threads (8 warps each); the ceiling division covers sizes that aren't multiples of the block size, so the kernel needs the `row < size && col < size` guard.

```

__global__ void matmul(float *A, float *B, float *C, int size) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < size && col < size) {
        float sum = 0.0f;
        for (int k = 0; k < size; k++)
            sum += A[row * size + k] * B[k * size + col];
        C[row * size + col] = sum;
    }
}

```

Memory behavior: `B[k*size+col]` is coalesced (adjacent threads, adjacent `col`); `A[row*size+k]` is the same address for all threads of a row (broadcast-friendly). But every thread re-reads a full row of A and column of B from global memory: $2n$ global loads per output with no reuse across the block. That is low arithmetic intensity and the slide's "speed this up" cue: **stage tiles of A and B in shared memory**, compute from there, write C once.

```

// My sketch of the tiled version (the slides only describe the idea)
#define TILE 16
__global__ void matmul_tiled(float *A, float *B, float *C, int n) {
    __shared__ float As[TILE][TILE], Bs[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;
    for (int t = 0; t < (n + TILE - 1) / TILE; t++) {
        int ac = t * TILE + threadIdx.x, br = t * TILE + threadIdx.y;
        As[threadIdx.y][threadIdx.x] = (row < n && ac < n) ? A[row * n + ac]
: 0.0f;
        Bs[threadIdx.y][threadIdx.x] = (br < n && col < n) ? B[br * n + col]
: 0.0f;
        __syncthreads();                // tile fully loaded before use
        for (int k = 0; k < TILE; k++) sum += As[threadIdx.y][k] * Bs[k]
[threadIdx.x];
        __syncthreads();                // finished using tile before
        overwrite
    }
    if (row < n && col < n) C[row * n + col] = sum;
}

```

Effect: each global element is loaded once per tile step and reused TILE times, cutting global traffic by about a factor of TILE (16x here).

Slide errors and ambiguities to verify

I re-derived the numbers on the slides. These are the places where I think the slide is wrong, loose, or hides an assumption. Where I'm relying on outside knowledge rather than the slides, it says so; check with your lecturer or tutorial notes before trusting my correction over the slide in an exam.

Lecture, slide	What the slide says	Issue	How to treat it
L06, 56 (bad coalescing)	Stride of 32 B: "128 total 32-byte requests", 12.5% utilization	32 threads each touching a separate 32-byte sector is 32 requests (1024 B fetched for 128 B used). 12.5% is right; "128" looks like a slip	Answer 32 sectors / 12.5%
L05, 39-40 (Gustafson)	$S_p = p / (1 + (p-1) f(n))$, limit p	This is Amdahl's expression with a size-dependent $f(n)$. The classic Gustafson scaled speedup is $S = p - \alpha (p - 1)$ where α is the serial share of the <i>parallel</i> run time	Use the slide's derivation for the course, know the classic form too
L06, 3	$f = 0.05$ needs more than 1024 PUs for 20x	Amdahl gives 19.6 at $p = 1024$; 20x is the asymptote and is never reached. 10x needs only $p = 19$	Quote the formula, not the rounded claim
L06, 32	A100 listed as CC 8.6	From my knowledge, A100 is CC 8.0 and 8.6 is the GA10x family (outside the slides, so verify)	Low stakes; irrelevant to computation
L06, 31	Hopper: 144 SMs, 96 GB, "H100"	144 SMs is the full GH100 die; shipped H100 SXM parts have 132 SMs and 80 GB (from my knowledge, verify)	Use the slide's per-SM figures for calculations

Lecture, slide	What the slide says	Issue	How to treat it
L06, 52	Pinned memory "is not cached"	Loose: pinned (page-locked) host memory is normally cacheable on the host; uncached applies to write-combined/zero-copy access patterns	Remember the point: pinned memory gives faster, async-capable transfers
L04, 46 (granularity)	8x8 grid: 512 vs 32 data transfers	Counts every task as having 4 neighbors and counts send plus receive (ignores grid boundaries, which would give 224 for the fine-grain case). Message count falls 16x but total data falls only about 4x (surface-to-volume)	Use the slide's convention in an exam; mention boundaries if asked to be precise
L04, 16-17	Degree of concurrency = total work / critical path	This is the average degree of concurrency; the textbook also defines the maximum degree (widest level of the graph)	State which one you compute
L02, 25 (pthreads)	Prints <code>iret1</code> , <code>iret2</code> as "Thread returns"	Those are <code>pthread_create</code> return codes, not thread results; the thread function also never returns a value, and <code>main()</code> has no return type	Don't copy this as a model; use <code>pthread_join(t, &result)</code> for results

Lecture, slide	What the slide says	Issue	How to treat it
L02, 56-57	Two producer-consumer versions	Both are correct. Signalling <code>items</code> after releasing <code>mutex</code> is only an efficiency improvement (avoids waking a consumer that immediately blocks). The version at slide 61 (<code>mutex</code> taken before <code>items.wait</code>) is the deadlocking one	Know the <i>reason</i> , not just the order
L03, 11	"Number of pipeline stages = maximum speedup"	True only for throughput of an ideal, hazard-free pipeline; single-instruction latency doesn't improve	Say "ideal throughput speedup"

What to learn beyond the slides

These gaps are either named on the slides without being taught, or are the obvious next question the slides raise. This list comes from my own knowledge of the subject, not from the lecture files; confirm what is in scope for your assessment.

Priority	Gap	Where the slides leave you hanging	What to learn
High	Dining philosophers and barbershop	L02 slide 54 lists them as classic problems, no solution is shown	Why the naive version deadlocks (all pick up the left fork: circular wait), and fixes: resource ordering, allow at most $n-1$ philosophers, asymmetric pickup. Barbershop: two semaphores plus a mutex for the waiting-room count

Priority	Gap	Where the slides leave you hanging	What to learn
High	Monitors and condition variables	L02 names them; L04 uses Java <code>wait / notify</code> without explanation	A monitor = mutex + condition variables; <code>wait</code> atomically releases the lock and sleeps; always re-check the condition in a <code>while</code> loop (spurious wakeups, Mesa semantics); <code>pthread_mutex_*</code> , <code>pthread_cond_*</code> , <code>pthread_barrier_*</code>
High	Deadlock handling in detail	L02 lists four approaches only	Prevention by lock ordering (breaks circular wait), wait-for graph cycle detection, banker's algorithm for avoidance
High	OpenMP specifics	L04 shows only <code>parallel for</code> with <code>shared / private</code>	<code>reduction(+:sum)</code> (the correct fix for a shared accumulator race), <code>`schedule(static</code>
High	MPI basics	L04 shows a master-worker skeleton before MPI is taught	<code>MPI_Send/Recv</code> (blocking, can deadlock if both sides send first), non-blocking <code>Isend/Irecv + Wait</code> , collectives (<code>Bcast</code> , <code>Scatter</code> , <code>Gather</code> , <code>Reduce</code> , <code>Allreduce</code>), rank/size idiom
High	Cache coherence protocols	L03 shows the problem, not the solution	Write-invalidate vs write-update, snooping (bus) vs directory-based (scales to NUMA), MESI states (Modified, Exclusive, Shared, Invalid) and what a write to a Shared line triggers

Priority	Gap	Where the slides leave you hanging	What to learn
High	False sharing	L05 shows it through the padding picture without the name or consequence	Two threads write different variables in one cache line; the line ping-pongs between cores (invalidations) so a "parallel" loop slows down; fix by padding/aligning per-thread data to the line size (typically 64 B on CPUs)
Medium	Memory consistency	L03 slide 50 mentions it, nothing more	Sequential consistency vs relaxed models, why compilers/CPU's reorder, fences, <code>volatile</code> is not synchronization; coherence = one location, consistency = order across locations
Medium	Atomic primitives beyond test-and-set	L02 stops at test-and-set	Compare-and-swap, fetch-and-add; test-and-test-and-set and exponential backoff to cut coherence traffic from spinning; spinlock vs blocking mutex break-even
Medium	NUMA in practice	L03 defines NUMA	First-touch page placement, thread pinning/ <code>numactl</code> , why remote memory access hurts, relation to the mapping step in Foster
Medium	Better scaling models	L05 gives Amdahl and Gustafson only	Karp-Flatt metric to infer the <i>experimental</i> serial fraction $e = (1/S - 1/p) / (1 - 1/p)$, isoefficiency (how fast n must grow to keep E constant), work-span model with the bound $T_p \geq \max(T_1 / p, T_\infty)$ (T_∞ = critical path, links to L04 TDG), Amdahl with overhead $1 / (f + (1-f)/p + \text{overhead}(p))$

Priority	Gap	Where the slides leave you hanging	What to learn
Medium	Roofline model	L05 arithmetic intensity is the x-axis of it	Attainable performance = $\min(\text{peak FLOPs}, \text{bandwidth} \times \text{arithmetic intensity})$; tells you whether a kernel is memory- or compute-bound without perturbation experiments
Medium	Parallel reduction and scan on GPU	L06 sums inside one thread only	Tree reduction in shared memory (use sequential addressing to avoid divergence and bank conflicts), warp shuffles (<code>__shfl_down_sync</code>), <code>atomicAdd</code> for global combine, prefix sum
Medium	CUDA practicalities	L06 omits them	Check every API return code (<code>cudaGetLastError</code>), time with CUDA events not CPU timers, streams for overlap, <code>cudaMemcpyAsync</code> needs pinned memory, profile with Nsight before optimizing
Low	Load balancing	L04 mentions static vs dynamic allocation	When tasks are irregular, dynamic pools beat static blocks; cost is synchronization on the pool

Where I'd push back on the framing

- **"Amdahl pessimistic, Gustafson optimistic" is a slogan, not a theorem.** They answer different questions (fixed work vs fixed time) and both ignore overheads that grow with p . Real scaling curves usually fall *below* both, so also reason about communication and synchronization costs.
- **GPU speedups are only as honest as the baseline.** L05 insists on the best sequential algorithm; a GPU result against an unoptimized single-thread CPU loop overstates the benefit. Include transfer time in the GPU total.

- **"The math is free" has limits.** Recomputing beats reloading only while the kernel stays memory-bound; once compute-bound, extra arithmetic costs time (the roofline model shows where the crossover sits).
- **More threads is not always faster**, on CPUs (oversubscription, contention) or GPUs (register spills at high occupancy). The slides say this; it is also the most common mistake in practice.

Suggested study order

1. Redo the Tier 1 items from the priorities table on paper from a blank page (semaphore patterns, Amdahl/Gustafson, TDG, occupancy, coalescing).
2. Close the High gaps above, starting with dining philosophers, OpenMP reduction, and MESI.
3. Do the self-test below without notes, then check answers.
4. Skim the readings the slides cite: *The Little Book of Semaphores* (Downey) for L02, Grama et al. *Introduction to Parallel Computing* for L04, and the CUDA C Programming/Best Practices guides for L06.

Self-test questions

Cover the answer column and work each one. I wrote these myself from the lecture content; they are practice, not past-paper questions.

#	Question	Answer
1	Producer-consumer: the consumer calls <code>mutex.wait()</code> before <code>items.wait()</code> . What goes wrong?	With an empty buffer the consumer holds the mutex and sleeps on <code>items</code> ; the producer needs the mutex to add an item, so neither proceeds. Deadlock (hold-and-wait, circular dependency)
2	Which of the four deadlock conditions does a global lock-ordering rule break?	Circular wait
3	Why does <code>while (held); held = 1;</code> fail as a lock, and how does test-and-set fix it?	Test and set are two steps; a context switch between them lets two threads both see 0. Test-and-set does read-modify-write indivisibly in hardware, so only one thread sees the old value 0

#	Question	Answer
4	In readers-writers with a lightswitch, who starves and how does a turnstile fix it? What does it cost?	Writers starve under a continuous stream of readers. A writer holds the turnstile while waiting for the room, so new readers queue behind it. Cost: readers are briefly delayed at the turnstile on every entry
5	Amdahl: $f = 0.2$, $p = 16$. Speedup, efficiency, limit?	$S = 1 / (0.2 + 0.8/16) = 4.0$; $E = 4/16 = 0.25$; limit $1/0.2 = 5$
6	$T_{seq} = 100$ s, $T_8 = 20$ s. S , E , cost? Is it cost-optimal?	$S = 5$, $E = 0.625$, cost = $8 \times 20 = 160$ s versus 100 s sequential. Not cost-optimal (cost is 1.6x the sequential work)
7	TDG: four leaf tasks of 10, two mid tasks of 6 (each takes two leaves), one root of 4 (takes both mids). Critical path, degree of concurrency?	Total = $40 + 12 + 4 = 56$; critical path = $10 + 6 + 4 = 20$; degree = $56/20 = 2.8$
8	Give a case where speedup exceeds p .	Per-core working set fits in cache once data is split, or threads share a warmed L3: the parallel run has fewer misses than the sequential run
9	AMAT: L1 hit 2 cycles, L1 miss rate 10%, L2 hit 12 cycles, L2 local miss rate 25%, memory 200 cycles.	$T_{L1miss} = 12 + 0.25 \times 200 = 62$; AMAT = $2 + 0.1 \times 62 = 8.2$ cycles; global miss rate 2.5%
10	Why does Gustafson's law not contradict Amdahl's?	Different question: Amdahl fixes the problem size (f constant, speedup capped); Gustafson fixes the time and grows the problem so $f(n)$ shrinks. Both assume no growing overheads
11	1000x1000 image, 16x16 blocks. Grid dimensions, total threads, wasted threads?	$\text{ceil}(1000/16) = 63$, so $63 \times 63 = 3969$ blocks; $1008^2 = 1,016,064$ threads; 16,064 are out of range and need the bounds guard
12	Warp reads <code>arr[2*i]</code> (floats, aligned) for thread i . Transactions and utilization?	32 threads span 256 B = 8 sectors; 128 B useful of 256 B fetched = 50%
13	Shared memory <code>s[threadIdx.x * 2]</code> vs <code>s[threadIdx.x * 33]</code> : bank conflicts?	Stride 2: 2-way conflict. Stride 33: $33 \bmod 32 = 1$, so every thread hits a different bank, no conflict

#	Question	Answer
14	Block of 512 threads, 40 registers/thread, limits 2048 threads and 65536 registers per SM. Occupancy?	Registers per block = 20480, so 3 blocks fit (3.2); thread limit would allow 4. 1536 threads = 48 warps of 64 = 75%
15	Which of these diverges in a warp: <code>if (threadIdx.x < 16)</code> or <code>if (threadIdx.x / 32 == 0)</code> ?	The first (half of warp 0 goes each way). The second is uniform per warp, so no divergence
16	Fine vs coarse partitioning of a grid: what do you gain and lose by agglomerating?	Gain: fewer messages, better surface-to-volume ratio, less task overhead. Lose: fewer tasks, worse load balance and less flexibility in mapping
17	Choose a pattern: a web crawler where new URLs are discovered while processing.	Task pool: tasks are generated dynamically, irregular work, fixed worker threads, synchronized pool
18	Why can a user-level threads library not exploit a multicore, and what else goes wrong?	The OS sees one process, so it schedules all user threads onto one execution resource; also one blocking I/O call blocks every thread
19	How would you find out whether a kernel is memory-bound?	Remove most of the math but keep the loads: little speedup means memory-bound. Also compute arithmetic intensity against the machine's bandwidth-to-compute ratio
20	A GPU kernel runs 3x faster than the CPU version but copying data takes longer than the CPU run. Is the GPU worth it?	Not as it stands: total time includes transfers. Batch/pin/overlap transfers, keep data resident on the device across kernels, or accept it only if more kernels will reuse the data

Sources

All content is derived from your five lecture slide decks: *L02 Processes, Threads, and Synchronization*; *L03 Parallel Computing Architectures*; *L04 Parallel Programming Models, Part I*; *L05 Performance of Parallel Systems*; *L06 GPU Computing: Architecture, Programming, and Performance*. Material under "What to learn beyond the slides", the tiled matmul code, the self-test questions, and the flagged slide corrections are my additions from general knowledge, not from the decks.